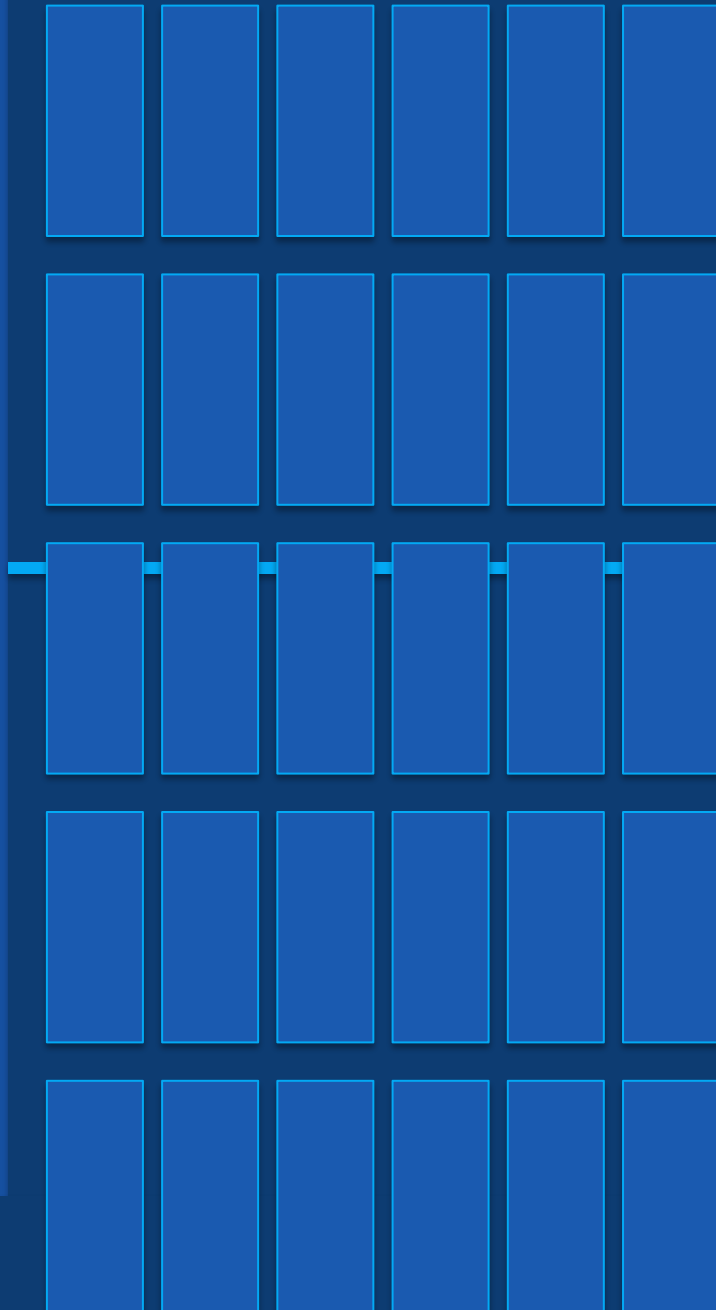


Pengembangan Aplikasi Web

Grid & Desain Responsif

Build adaptive, mobile-first layouts using CSS Grid and Material Design principles



Agenda

Yang akan kita bahas hari ini

01

Apa itu CSS Grid?

Konsep dasar dan terminologi

03

Prinsip Desain Responsif

Mobile-first, breakpoint, tata letak fluida

05

Grid vs Flexbox

Kapan menggunakan alat t layout

02

Pendalaman Properti Grid

properti container & item

04

Sistem Grid Material Design

Sistem 12 kolom, spasi, gutter

06

Examples

Dashboard, card layout, form layout

07

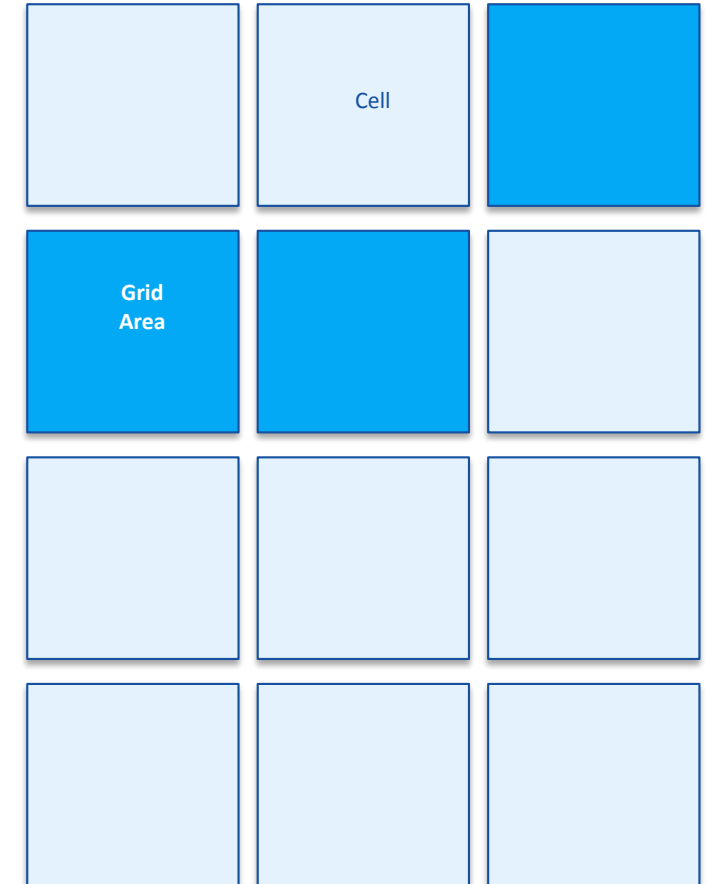
Apa itu CSS Grid?

Sistem tata letak dua dimensi untuk web

CSS Grid Layout adalah sistem tata letak dua dimensi yang powerful, memungkinkan mengatur konten dalam baris AND kolom secara bersamaan. Berbeda dengan Flexbox yang satu dimensi.

Grid Container	Elemen induk dengan display: grid
Grid Item	Child langsung dari grid container
Garis Grid	Garis pembagi yang membentuk struktur grid
Track Grid	Ruang antara dua garis grid yang berdekatan (baris atau kolom)
Sel Grid	Single unit - perpotongan track baris dan kolom
Area Grid	Ruang persegi panjang yang dikelilingi empat garis grid

Grid Anatomy



Properti Grid Container

Diterapkan pada elemen induk (display: grid)

code

```
.container {
  display: grid;

  /* Define columns */
  grid-template-columns: 1fr 2fr 1fr;
  /* or: repeat(3, 1fr) | 200px auto 1fr */

  /* Define rows */
  grid-template-rows: 80px auto 120px;

  /* Shorthand: rows / columns */
  grid-template: 80px auto / repeat(3, 1fr);

  /* Gap between tracks */
  gap: 16px;          /* row-gap column-gap */
  column-gap: 24px;
  row-gap: 16px;

  /* Alignment */
  justify-items: stretch; /* start|end|center */
  align-items: center;
  place-items: center stretch;
}
```

grid-template-columns

Mendefinisikan track kolom. Gunakan px, %, fr, auto, repeat(), minmax().

grid-template-rows

Mendefinisikan tinggi track baris eksplisit.

gap / row-gap / column-gap

Jarak antar track grid (menggantikan grid-gap lama).

grid-auto-rows

Ukuran baris yang dibuat secara implisit (mis., item yang ditempatkan otomatis).

grid-auto-flow

Mengontrol penempatan otomatis: row, column, dense.

justify-content

Meratakan seluruh grid secara horizontal di dalam container.

align-content

Meratakan seluruh grid secara vertikal di dalam container.

justify-items / align-items

Meratakan semua item grid di dalam sel mereka.

Property Grid Item

Diterapkan pada children dari grid container

code

```
.item {
  /* Placement by line numbers */
  grid-column-start: 1;
  grid-column-end: 3;    /* span to line 3 */
  grid-row-start: 1;
  grid-row-end: 2;

  /* Shorthand */
  grid-column: 1 / 3;    /* start / end */
  grid-row: 1 / 2;

  /* Span keyword */
  grid-column: 1 / span 2;
  grid-row: span 3;

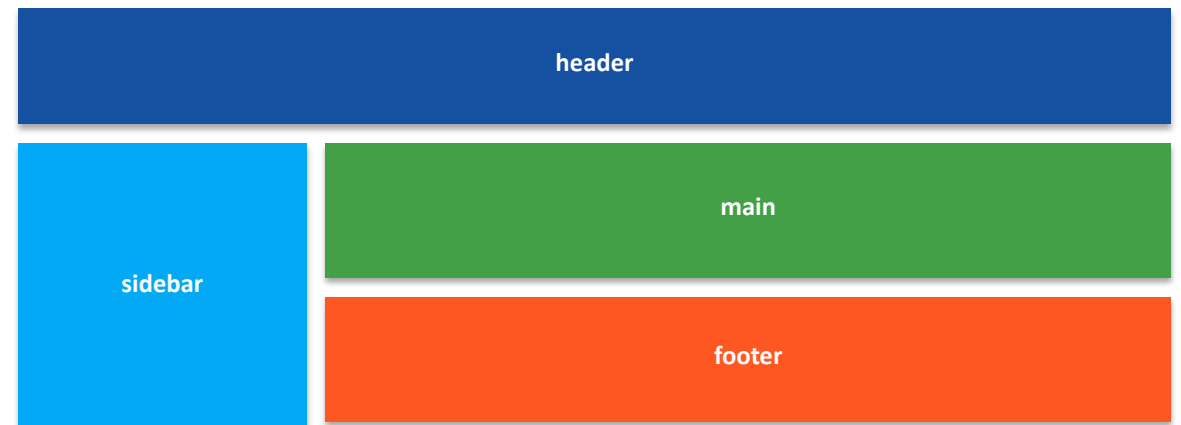
  /* Named area */
  grid-area: header;

  /* Self-alignment (overrides container) */
  justify-self: center;
  align-self: end;
  place-self: center end;
}
```

Contoh grid-template-areas

code

```
.layout {
  grid-template-areas:
    "header header header"
    "sidebar main    main  "
    "sidebar footer footer";
  grid-template-columns: 200px 1fr 1fr;
  grid-template-rows: 60px 1fr 50px;
}
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.footer  { grid-area: footer; }
```



Prinsip Desain Responsif

Mobile-first, fluid layouts, breakpoints

1. Mobile-First

Mulai menata style untuk layar terkecil. Gunakan media query min-width untuk meningkatkan tampilan secara progresif. Style default = mobile.

3. Gambar Fleksibel

Gunakan max-width: 100% pada gambar agar tidak meluap dari container. Gunakan object-fit untuk kontrol.

5. Viewport Meta

`<meta name="viewport" content="width=device-width, initial-scale=1">` WAJIB ada untuk perilaku responsif di mobile.

2. Grid Fluida

Gunakan satuan relatif (fr, %, em) alih-alih piksel tetap. Kolom harus fleksibel dan berkembang sesuai lebar viewport.

4. Media Query

Terapkan perubahan tata letak pada breakpoint. Breakpoint umum: 480px (sm), 768px (md), 1024px (lg), 1280px (xl).

6. Prioritas Konten

Pastikan konten penting muncul pertama dalam HTML (urutan DOM = urutan screen reader / mobile). Gunakan CSS untuk mengatur ulang secara visual.

Breakpoints & Media Queries

Mendefinisikan perubahan tata letak di berbagai ukuran layar

code

```
/* === MOBILE FIRST APPROACH === */

/* Base (mobile) - no query needed */
.grid {
  display: grid;
  grid-template-columns: 1fr; /* 1 col */
  gap: 16px;
}

/* Small tablets ≥ 480px */
@media (min-width: 480px) {
  .grid { grid-template-columns: repeat(2, 1fr); }
}

/* Tablets ≥ 768px */
@media (min-width: 768px) {
  .grid { grid-template-columns: repeat(3, 1fr); }
}

/* Desktops ≥ 1024px */
@media (min-width: 1024px) {
  .grid { grid-template-columns: repeat(4, 1fr); }
}

/* Wide screens ≥ 1280px */
@media (min-width: 1280px) {
  .grid { grid-template-columns: repeat(6, 1fr); }
}
```

Breakpoint Umum

xs

< 480px

Sangat Kecil • ponsel

sm

≥ 480px

Kecil • ponsel besar

md

≥ 768px

Sedang • tablet

lg

≥ 1024px

Besar • desktop

xl

≥ 1280px

Sangat Besar • layar lebar

Sistem Grid Material Design

Grid 12 kolom, margin, gutter & kolom

Material Design menggunakan sistem grid 12 kolom yang beradaptasi di berbagai breakpoint. Tata letak didefinisikan oleh margin (ruang di sisi), gutter (ruang antar kolom), dan kolom.

Breakpoint	Ukuran Jendela	Kolom	Margin	Gutter
Sangat Kecil (xs)	< 600dp	4	16dp	16dp
Kecil (sm)	600–904dp	8	32dp	16dp
Sedang (md)	905–1239dp	12	auto	24dp
Besar (lg)	1240–1439dp	12	200dp	24dp
Sangat Besar (xl)	≥ 1440dp	12	auto	24dp

Grid Lanjutan: auto-fit & minmax()

Grid responsif TANPA media query

Pola CSS Grid responsif yang paling powerful, buat grid fluida yang beradaptasi sendiri dan menyesuaikan jumlah kolom secara otomatis berdasarkan ruang yang tersedia.

code

```
/* Auto-fit: fills row, collapses empty tracks */
.auto-fit-grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(250px, 1fr));
  gap: 24px;
}

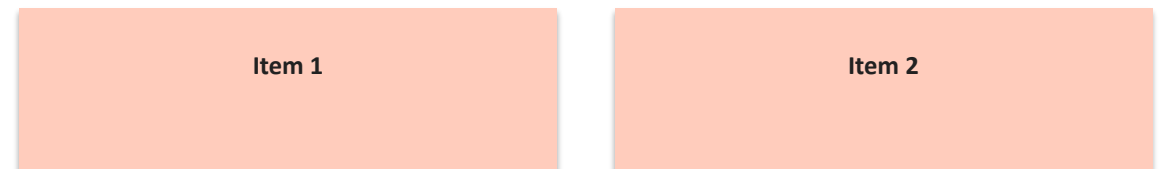
/* Auto-fill: reserves space for empty tracks */
.auto-fill-grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(200px, 1fr));
  gap: 16px;
}

/* minmax(min, max)
   min = smallest the column can be
   max = largest (1fr = remaining space)
   auto-fit = collapse empty columns
   auto-fill = keep empty column slots */
```

Layar lebar (4 kolom)



Layar sempit (2 kolom — otomatis)



Grid vs Flexbox

Memilih layout tool yang tepat

CSS Grid

Dua Dimensi

Mengontrol baris DAN kolom secara bersamaan

Berbasis Tata Letak

Definisikan tata letak terlebih dahulu, lalu tempatkan item ke dalamnya

Penempatan Eksplisit

Kontrol presisi atas posisi setiap item

Area Grid

Beri nama wilayah dan tetapkan item ke dalamnya

Gunakan untuk:

Tata letak halaman, dashboard, grid kartu, form kompleks

VS

Flexbox

Satu Dimensi

Bekerja sepanjang satu sumbu (baris ATAU kolom)

Berbasis Konten

Item mendefinisikan berapa banyak ruang yang mereka ambil

Ukuran Dinamis

Item berkembang/menyusut berdasarkan konten & ruang

Perataan

Sangat baik untuk meratakan item dalam baris/deretan

Gunakan untuk:

Navigation bar, grup tombol, baris form, pemusatan

Contoh : Tata Letak Dashboard

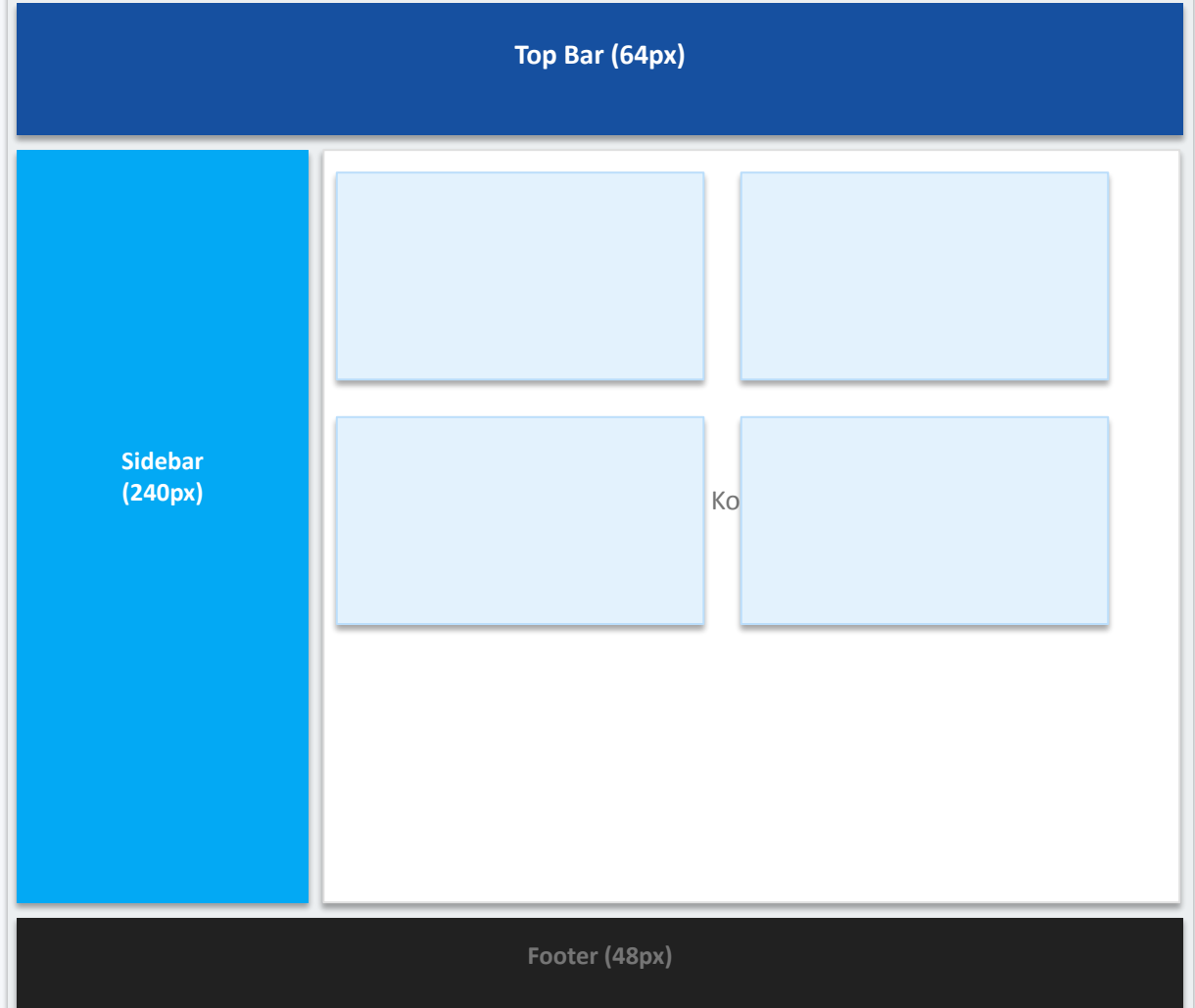
Menggunakan area grid bernama untuk shell aplikasi

code

```
/* Dashboard layout */
.app {
  display: grid;
  min-height: 100vh;
  grid-template-areas:
    "topbar topbar"
    "sidebar content"
    "footer footer";
  grid-template-columns: 240px 1fr;
  grid-template-rows: 64px 1fr 48px;
}
.topbar { grid-area: topbar; }
.sidebar { grid-area: sidebar; }
.content { grid-area: content; }
.footer { grid-area: footer; }

/* Responsive collapse */
@media (max-width: 768px) {
  .app {
    grid-template-areas:
      "topbar"
      "content"
      "footer";
    grid-template-columns: 1fr;
  }
  .sidebar { display: none; }
}
```

Pratinjau Visual



Contoh: Grid Kartu Responsif

auto-fit + minmax untuk kartu produk/blog yang fluida

code

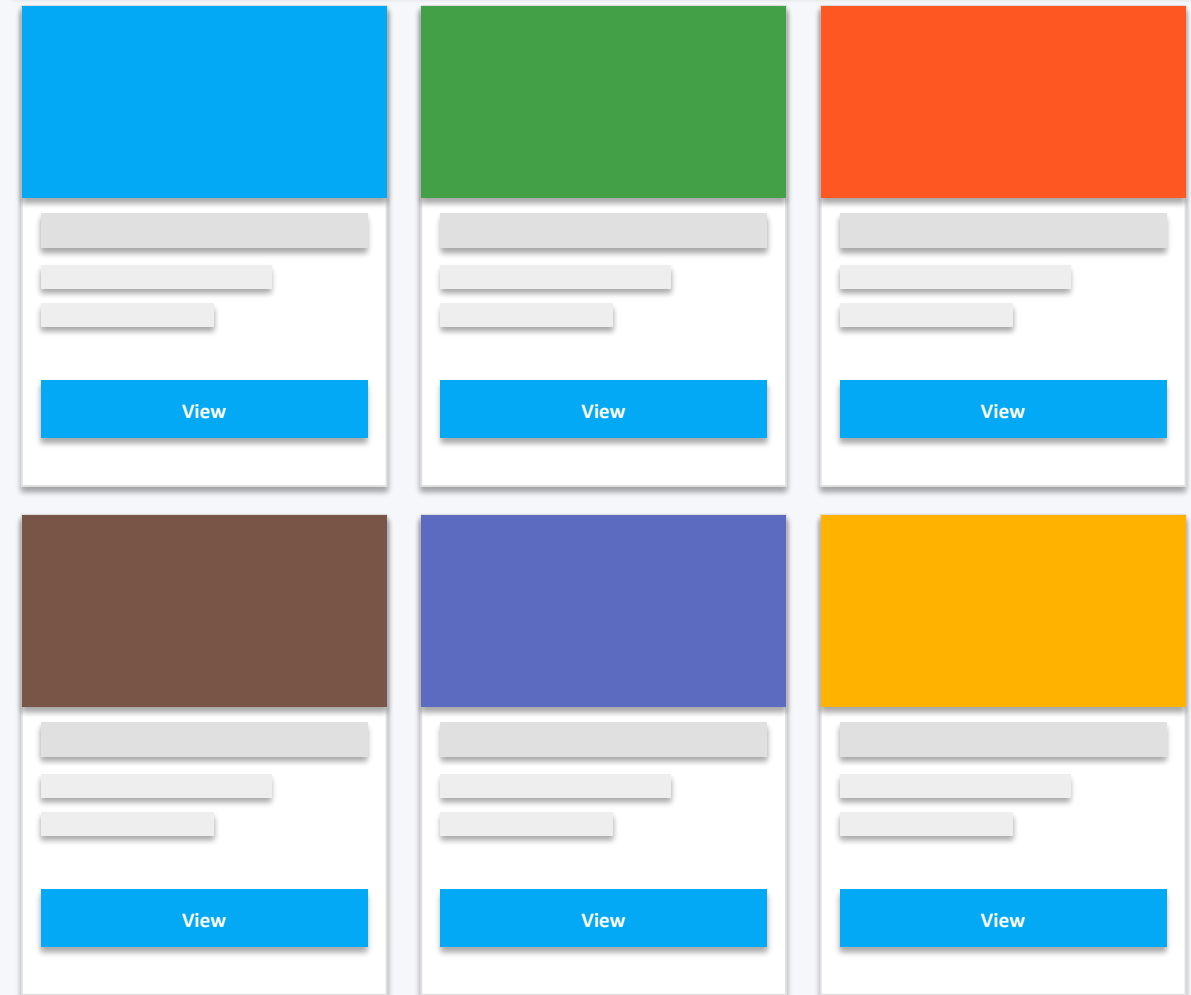
```
/* HTML */
<div class="card-grid">
  <div class="card">...</div>
  <div class="card">...</div>
  <!-- more cards -->
</div>

/* CSS */
.card-grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(280px, 1fr));
  gap: 24px;
  padding: 24px;
}

.card {
  background: white;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,.12);
  overflow: hidden;
  display: grid;
  grid-template-rows: 180px auto 1fr auto;
}

.card-image { object-fit: cover; }
.card-title { padding: 16px 16px 8px; }
.card-body { padding: 0 16px; }
.card-footer { padding: 16px; }
```

Desktop (3 kolom)



<https://github.com/tazkia-tech/learning-web-development.git>